PlanOwl: Automated PDDL Files Generation from OWL Ontologies and Visual Language Models

Mark Adamik, Paolo Forte

VU VRIJE UNIVERSITEIT AMSTERDAM

KCAI Knowledge in Artificial Intelligence

ÖREBRO UNIVERSITET

AAAI-FSS UR-RAD
06-11-2025

Problem Domain Definition Language (PDDL)

What is PDDL:

Formal language to describe automated planning problems

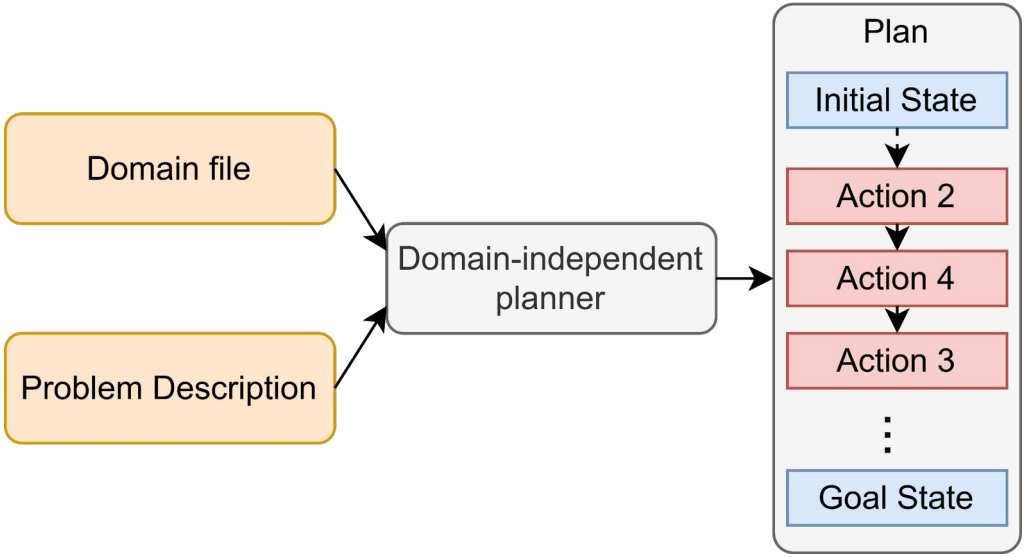
Components:

Domain file defines:

- Types
- Predicates
- Actions

Problem file specifies:

- Objects
- Initial state
- Goal state



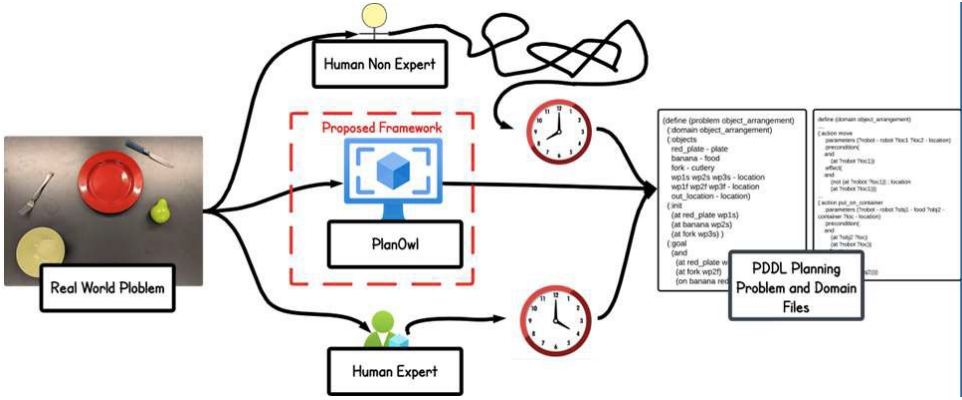
Problem Domain Definition Language (PDDL)

Why use it?

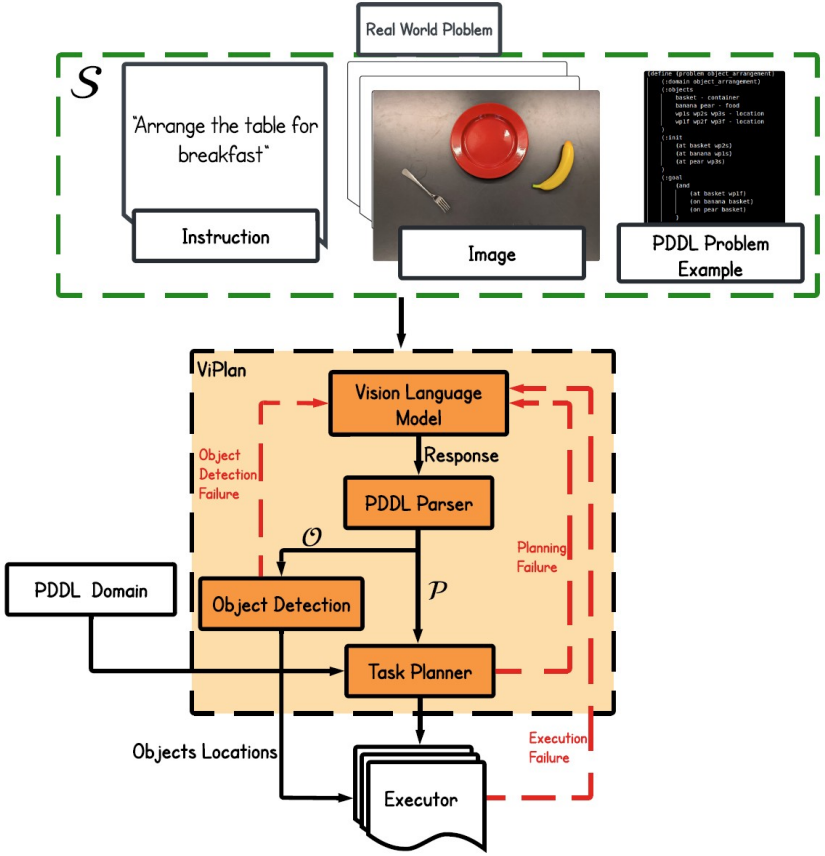
- Formal guarantees (plan correctness, goal satisfaction)
- Transparent logic and symbolic reasoning
- Fast once model is set up

Problems with it:

- Hard-coded and time-consuming
- Rigid abstractions: Can’t handle uncertainty
- Limited semantics: No ontological grounding



Previous Work - ViPlan



Source: V Aregbede et al. :Here's your PDDL Problem File! On Using VLMs for Generating Symbolic PDDL Problem Files (ICRA 2025)

Previous Work - ViPlan

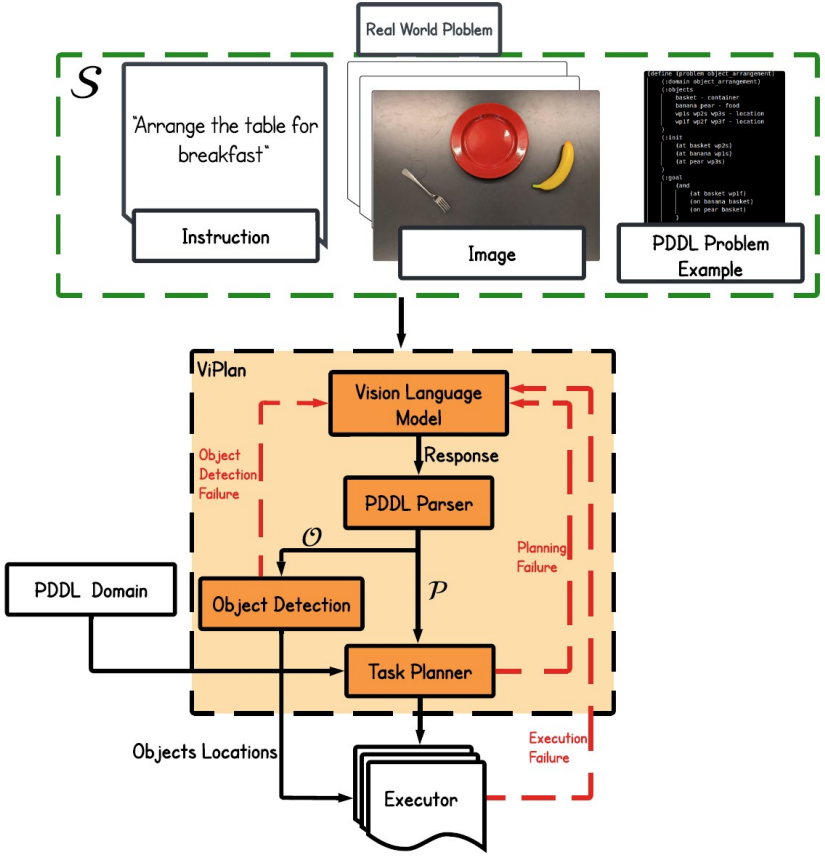
Problems with PDDL

- Rigid abstractions

  → VLMs can interpret images and new situations

- Hard-coded & Hand-crafted

  → Automate the generation process



Source: V Aregbede et al. :Here's your PDDL Problem File! On Using VLMs for Generating Symbolic PDDL Problem Files (ICRA 2025)

Previous Work - ViPlan

Problems with PDDL

- Rigid abstractions

  → VLMs can interpret images and new situations

- Hard-coded & Hand-crafted

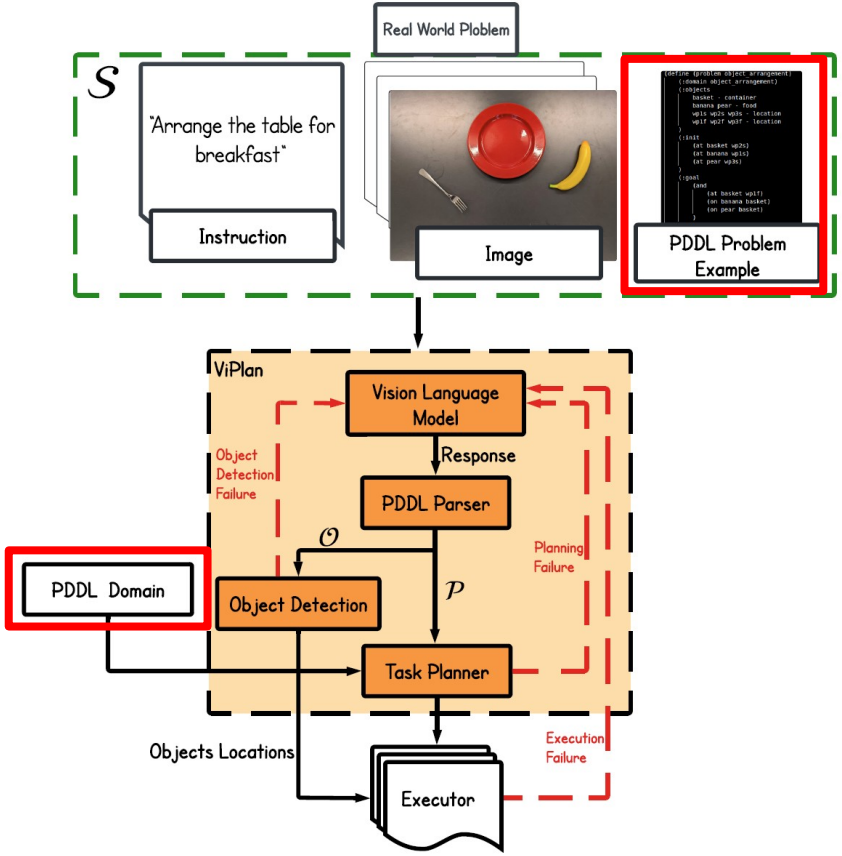
  → Automate the generation process

Limitations of ViPlan

△ - Hard-coded and hand-crafted:

  - PDDL Domain
  - PDDL Problem Example

- Limited semantics: No ontological grounding



Source: V Aregbede et al. :Here's your PDDL Problem File! On Using VLMs for Generating Symbolic PDDL Problem Files (ICRA 2025)

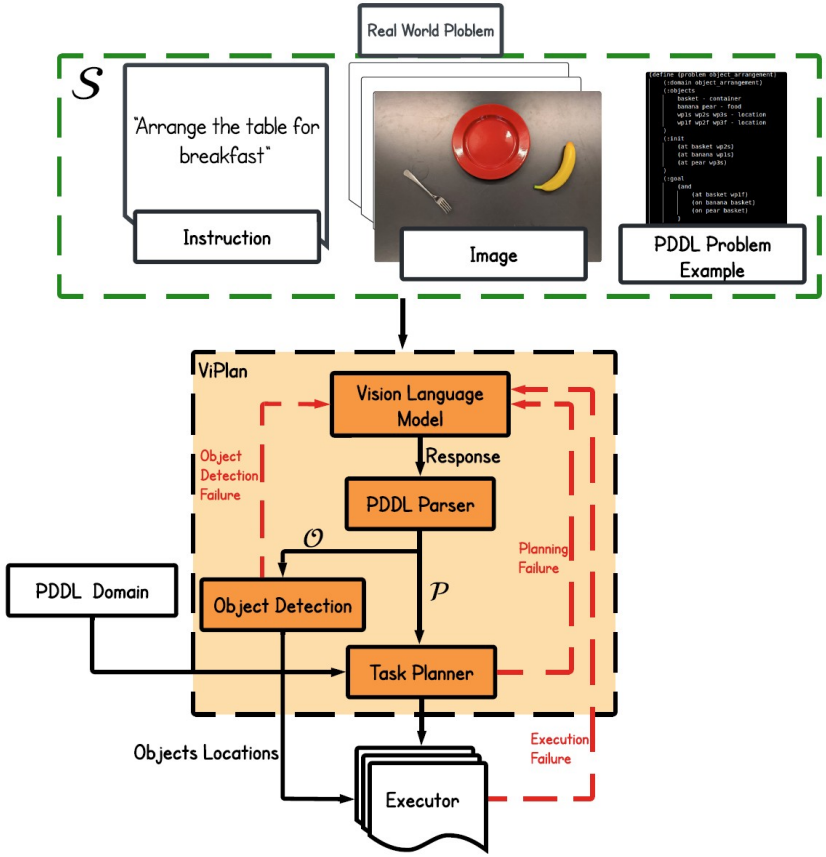
Research Questions

Research Problem

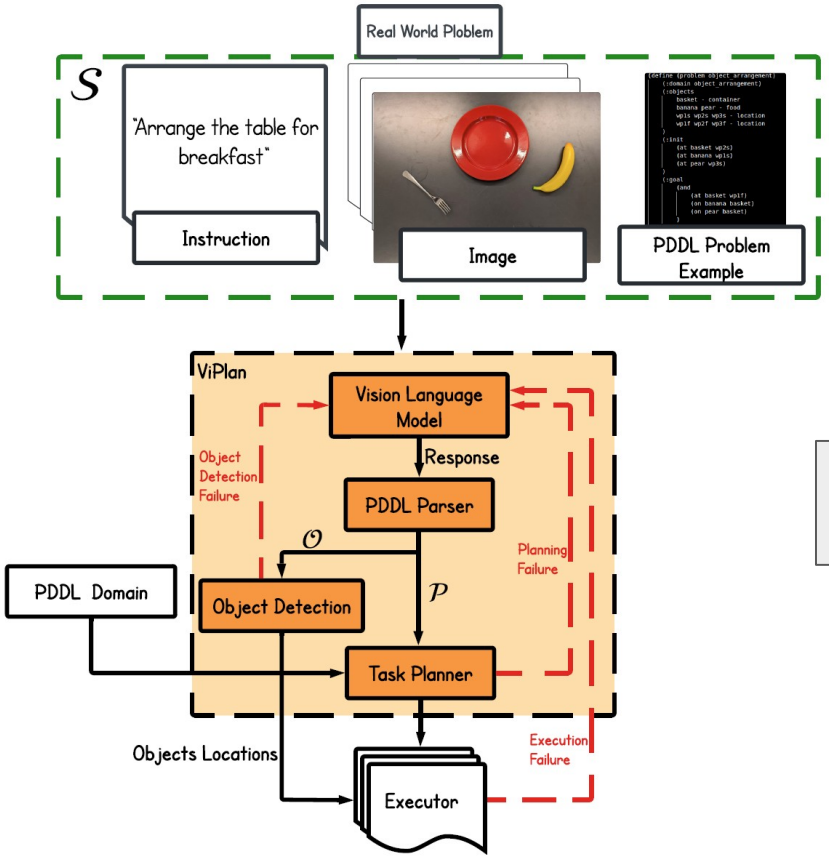
> *How can we **automatically generate both PDDL domain** and **problem files** for robotic task planning while **providing an ontological grounding** and utilizing the perceptual capabilities of Visual Language Models (VLMs)?*

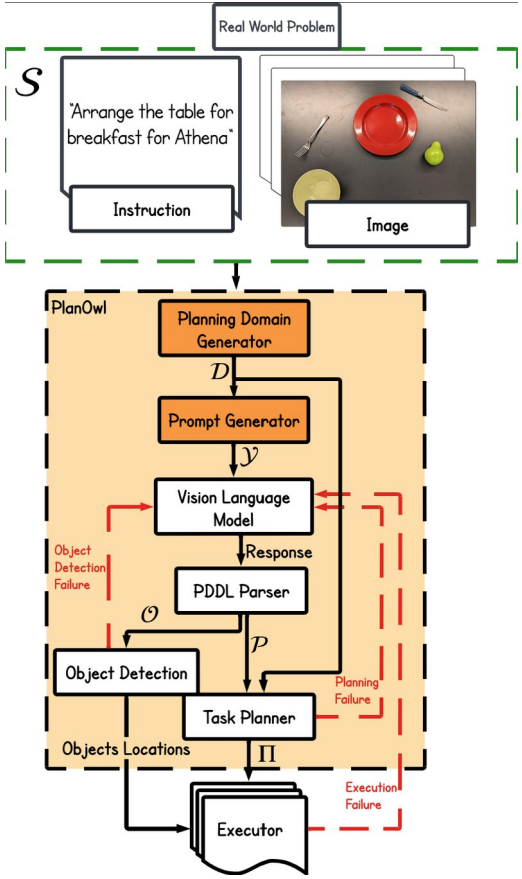
System Design

Approach


Real World Ploblem
S
"Arrange the table for breakfast"
Instruction
Image
PDDL Problem Example
ViPlan
Vision Language Model
Response
PDDL Parser
$\mathcal{O}$
$\mathcal{P}$
Object Detection Failure
Planning Failure
PDDL Domain
Object Detection
Task Planner
Objects Locations
Execution Failure
Executor

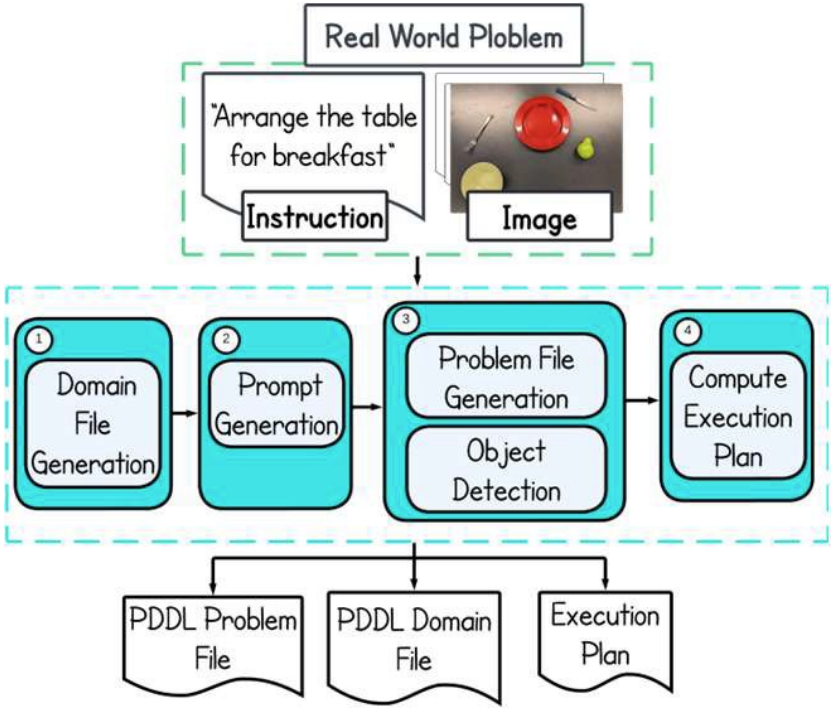
Approach


Real World Ploblem
S
"Arrange the table for breakfast"
Instruction
Image
PDDL Problem Example
ViPlan
Vision Language Model
Response
PDDL Parser
Object Detection Failure
Planning Failure
PDDL Domain
Object Detection
Task Planner
Objects Locations
Execution Failure
Executor


Real World Problem
S
"Arrange the table for breakfast for Athena"
Instruction
Image
PlanOwl
Planning Domain Generator
Prompt Generator
Vision Language Model
Response
PDDL Parser
Object Detection Failure
Planning Failure
Object Detection
Task Planner
Objects Locations
Execution Failure
Executor

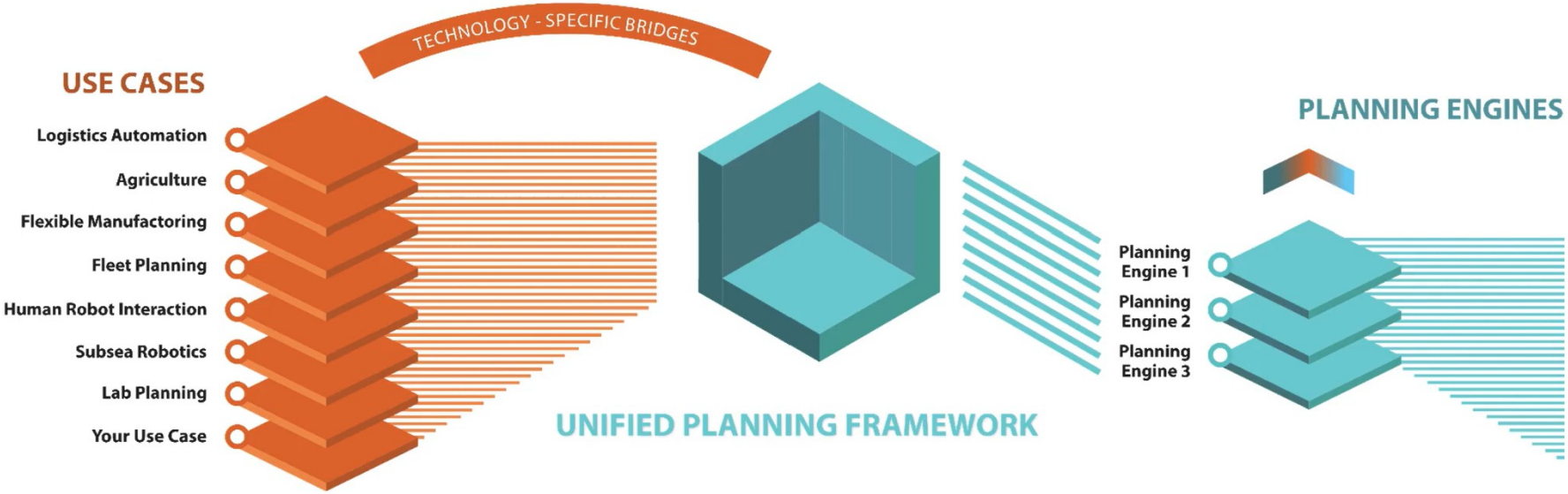
Approach - Overview

- **(1) Domain Generator:** Selects and reconstructs domain from ontolog.
- **(2) Prompt Generator:** Builds a context-aware prompt using domain + instruction.
- **(3) VLM + Object Detection:** Extracts initial and goal states from image + prompt.
- **(4) Planner:** Computes executable plan using generated PDDL file.
- **(5) Behavior Tree:** Handles failures, reprompts VLM, updates plan.



System elements - Unified Planning Framework

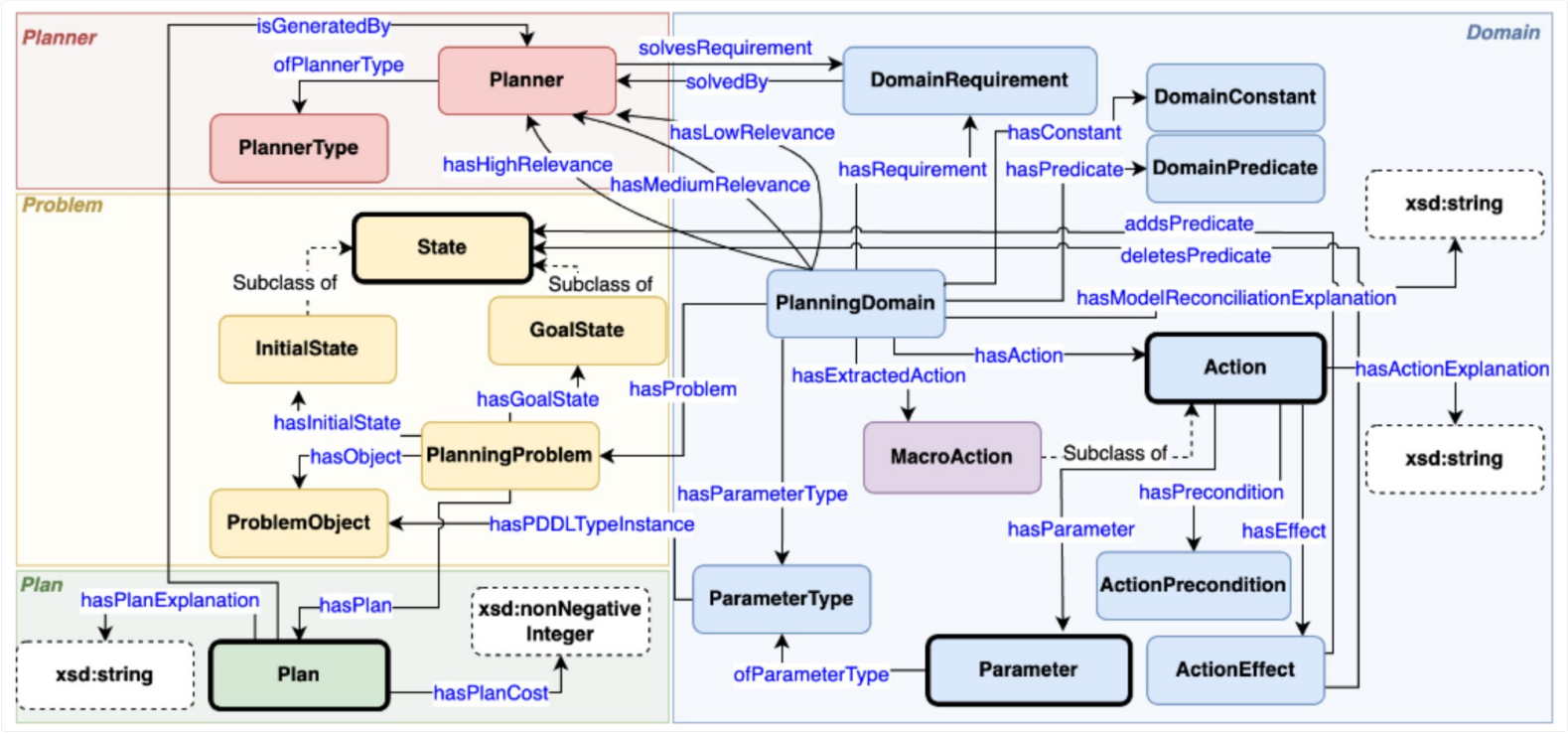
A Python-based library for modeling, solving, and integrating AI planning problems.



Source: Micheli et al.: Unified Planning: Modeling, manipulating and solving AI planning problems in Python (SoftwareX, 2025)

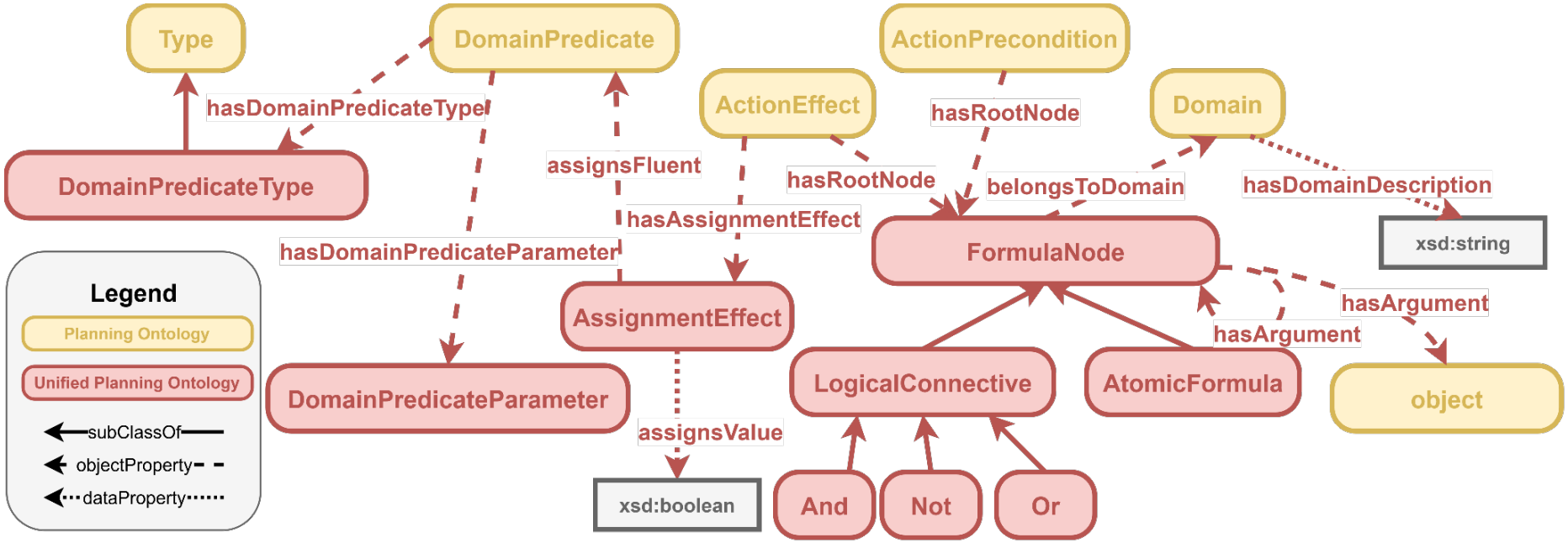
Image Source: https://unified-planning.readthedocs.io/en/latest/index.html

System elements - Planning Ontology



Source: B. Muppasani et al.: Building and Using a Planning Ontology from Past Data for Performance Efficiency (PLATO, ICAPS 2023)

Unified Planning Ontology



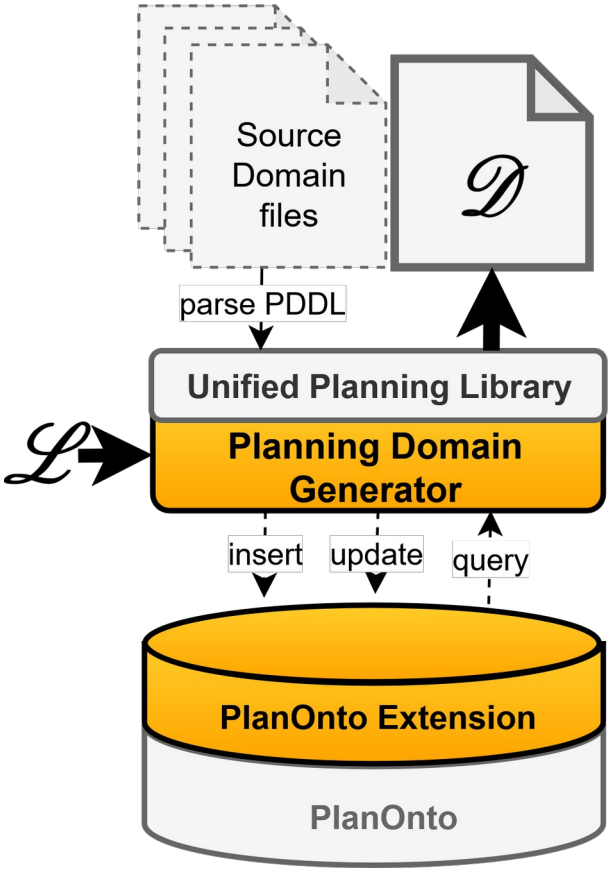
Novel elements Combined

1. **Setup:** Source Domain Files are parsed and inserted into PlanOnto

2. **Input**: Natural language Instruction

3. PlanOnto is queried for the closest domain (using the `hasDomainDescription predicate`) matching the instruction

4. The returned domain is re-constructed and inserted into the prompt



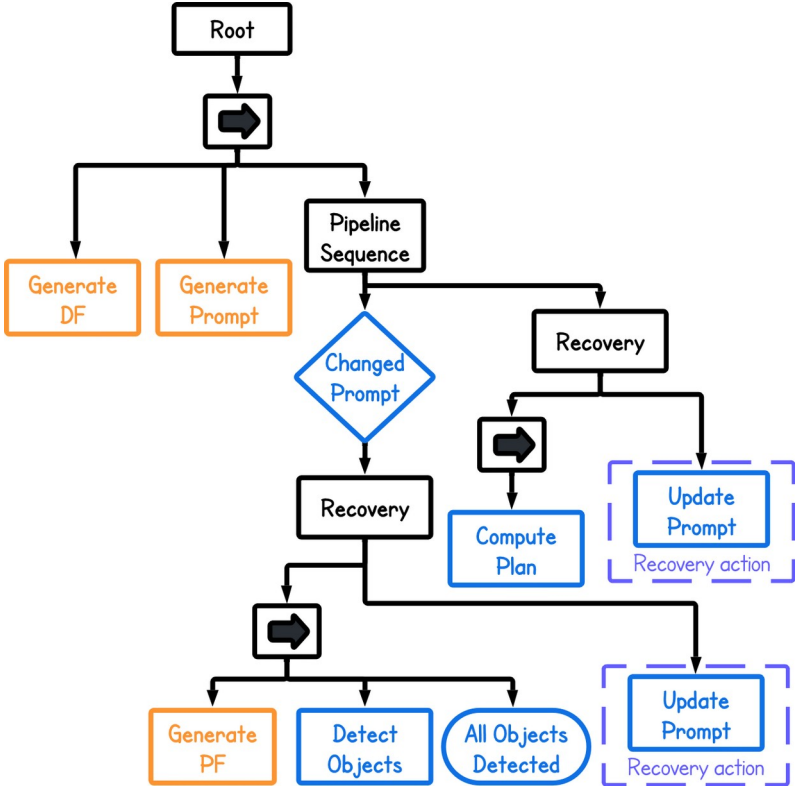
Extending the behaviour tree

New Extensions:

- Generate Domain File
- Generate Prompt
- Generate Problem File

Failure Recovery Handlers:

- **Object Detection Failure** → Retry sensing
- **Planning Failure** → Reprompt VLM with updated context



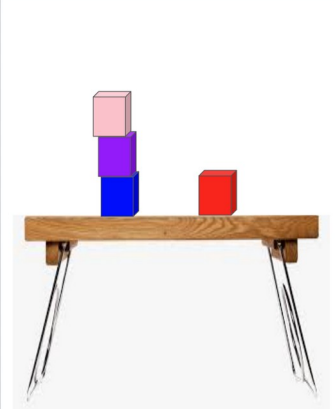
Results

Experiment Setup

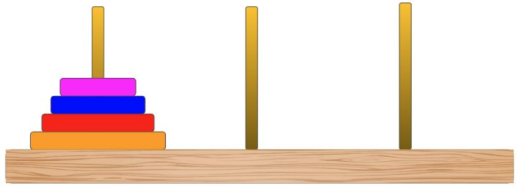
VLM used: Gemini 2.5 Pro
Planner: Fast Downward planner
Object Detection: GroundedSAM2

Benchmark Problems:

BlocksWorld

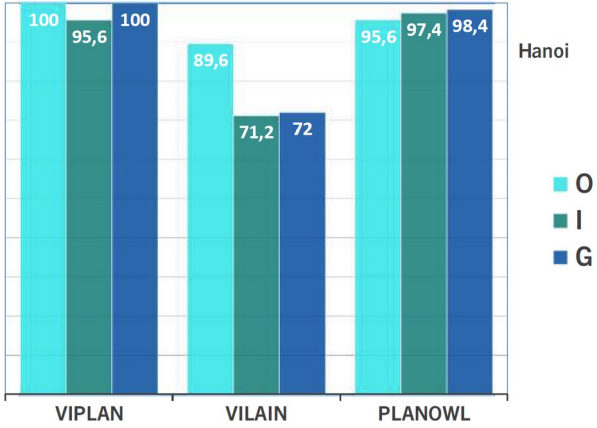


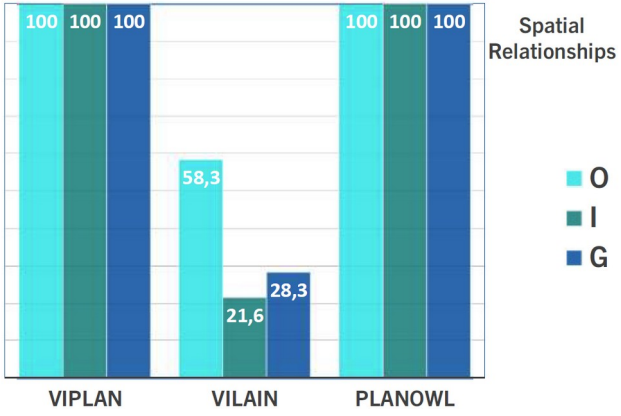
Tower of Hanoi



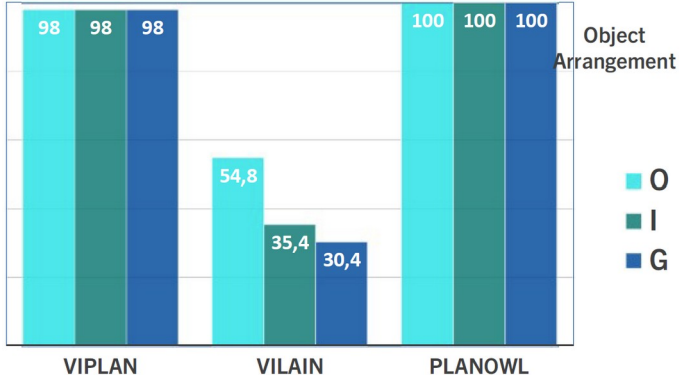
Object Arrangement

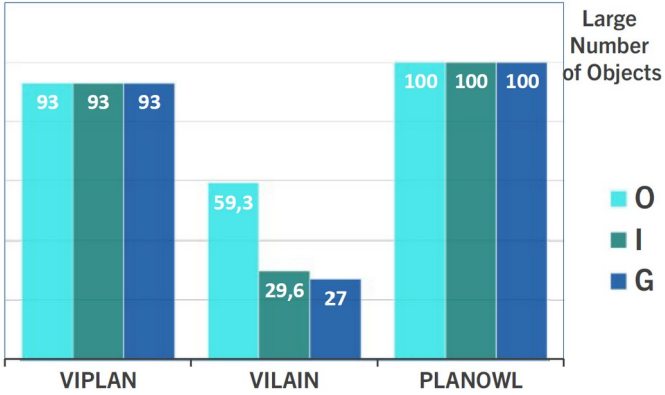
Results across domains





O = Object List
I = Initial conditions
G = Goal conditions



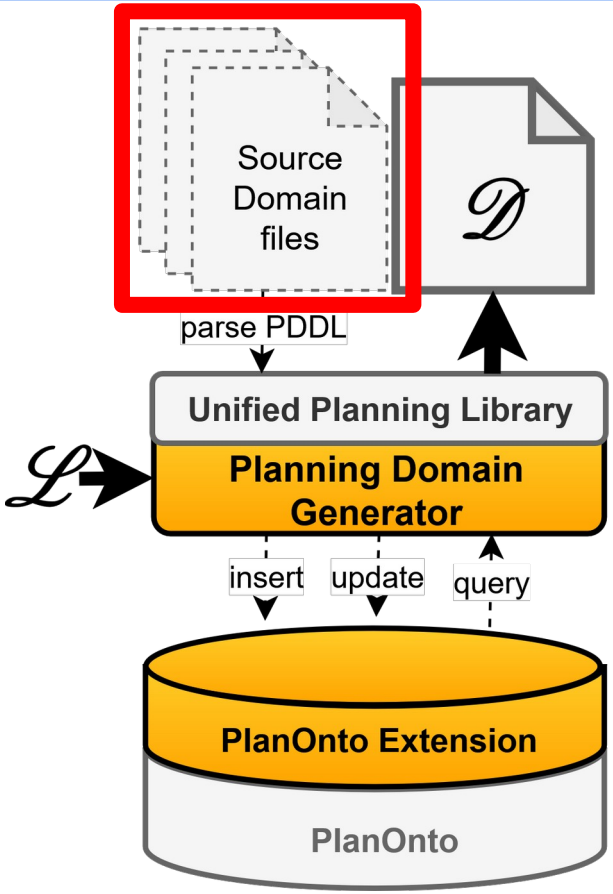


Next Steps

Limitations & Future Work

1. A collection of **Domain files** are required to populate the ontology
2. Evaluation is limited to three domains

Future Work: *Can we generate the PDDL domain files purely from the Natural Language Instructions?*



Challenges

Domain

```
(define (domain object_arrangement)

(:requirements :strips :typing :fluents :negative-preconditions)

(:types
    location - object
    cutlery - object
    container - object
    plate - object
    food - object
    robot - object
    glass - object
    mug - object
    plate - object
    screwdriver - object
    bowl - object
    fruit - food
    tool - object
    cup - object
    basket - object

(:predicates
    (free ?obj - object)
    (at ?obj - object ?loc - location)
    (on ?obj1 - food ?obj2 - object)
    (in ?obj1 - food ?obj2 - container)
)

(:action move
    :parameters (?robot - robot ?loc1 ?loc2 - location)
    :precondition(
    and
        (at ?robot ?loc1)
    )
    :effect(
    and
        (not (at ?robot ?loc1)) ; location
        (at ?robot ?loc1)
    )
)

(:action pick
    :parameters (?robot - robot ?obj - object ?loc - location)
    :precondition(
    and
        (free ?robot)
        (free ?obj)
        (at ?obj ?loc)
        (at ?robot ?loc)
    )
    :effect(
    and
        (not (free ?robot)) ; robot
        (not (at ?obj ?loc)) ; location
    )
)

(:action place_on_table
    :parameters (?robot - robot ?obj - object ?loc - location)
    :precondition(
    and
        (not (free ?robot))
        (at ?robot ?loc)
    )
    :effect(
    and
        (free ?robot)
        (at ?obj ?loc)
    )
)


(:action remove_from_table
    :parameters (?robot - robot ?obj - object ?loc - location)
    :precondition(
    and
        (not (free ?robot))
        (at ?robot ?loc)
    )
    :effect(
    and
        (free ?robot)
        (at ?obj ?loc)
)
)

(:action put_on_plate
    :parameters (?robot - robot ?obj1 - food ?obj2 - plate ?loc - location)
    :precondition(
    and
        (at ?obj2 ?loc)
        (at ?robot ?loc)
    )
    :effect(
    and
        (free ?robot)
        (on ?obj1 ?obj2)
    )
)

(:action put_in_container
    :parameters (?robot - robot ?obj1 - food ?obj2 - container ?loc -
location)
    :precondition(
    and
        (at ?obj2 ?loc)
        (at ?robot ?loc)
    )
    :effect(
    and
        (free ?robot)
        (on ?obj1 ?obj2)
    )
)
```

Types The objects involved in the specific domain.

Approach: It should be predicted from the sensory input, the natural language instruction and general domain knowledge (Common-Sense Knowledge Graph)

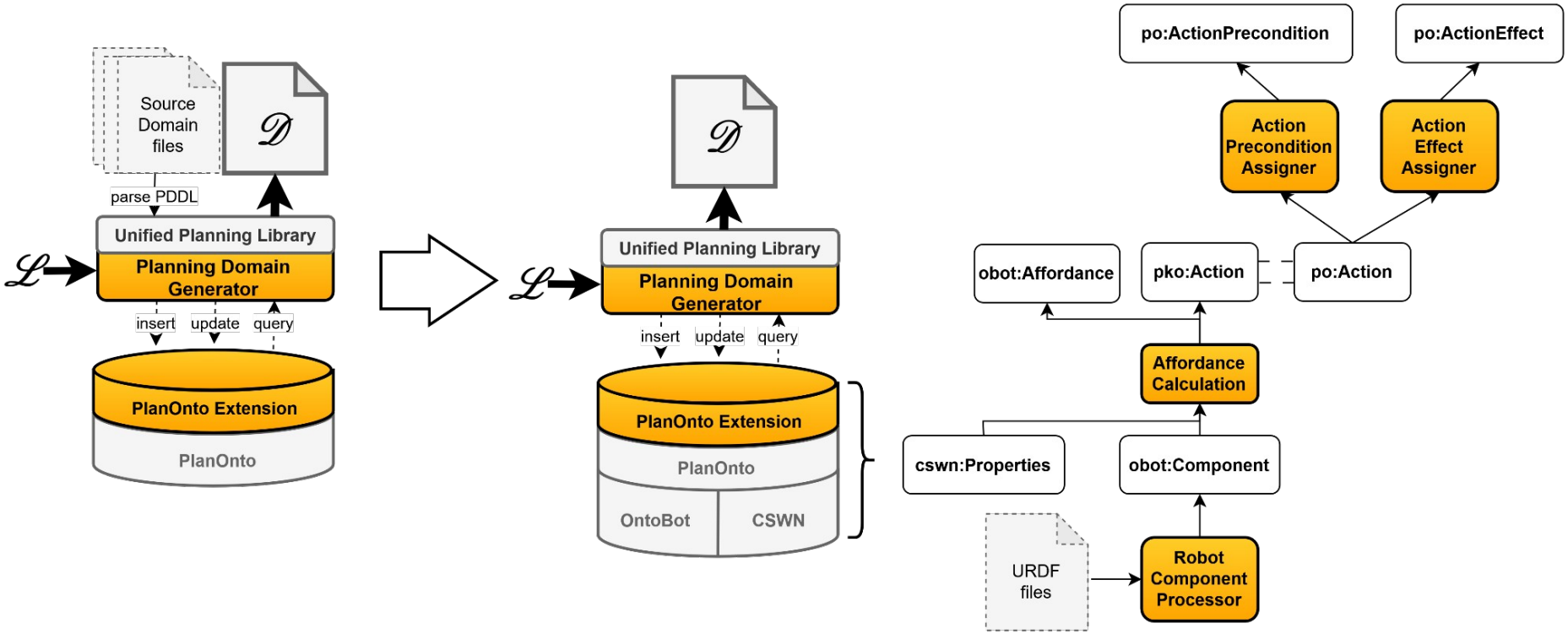
Predicates Usually common across domains.

Approach: Sourced by automatically parsing PDDL files (using UniPlan and IPC challenge as source), and collecting and categorizing these predicates (link prediction)

Actions: Actions the robot's component and the environment affords.

Approach: Construction from a collection of sources (Commmonsense Knowledge Graph and Robot Description sources)

Automatic PDDL Domain Construction



Conclusion

Summary

*How can we **automatically generate both PDDL domain** and **problem files** for robotic task planning while **providing an ontological grounding** and utilizing the perceptual capabilities of Visual Language Models (VLMs)?*

Approach:

- Combining Ontologies for enrichment and reusability with VLM-based planning systems

Contributions:

- Interface between an extended PlanOnto ontology and the Unified Planning Framework
- Extension of ViPlan

(Current) Future Work:

- Extend PlanOwl with more general automatic domain construction

Thank you!

m.adamik@vu.nl

https://github.com/PaoloForte95/metis